# 作业 A · 从深度二延伸到深度三：织成网，答全景题

## 一、为什么必须升级（问题本身）

深度二的向量 RAG 在单公司题上很好（8/10 全对），但两道跨公司全景题只覆盖 **2/12 家**。

原因不是"检索不够强"，而是**窗口只有 6 块**，无法容纳 12 家公司公司的数字；

而那些数字之间的**关系**（谁高谁低、谁在应对同一个行业问题），不写在任何一块里。

复习课程讲义那句判据：**要不要升级成知识图谱，看的不是库有多大，是答案是不是散在文档之间的关系里。**

## 二、按课程规定的六步建图（全部真跑）

| 步骤 | 做法 | 花的钱 | 产物 |
|---|---|---|---|
| ① 切块/选材 | 12 家 × 50 块（管理层讨论的叙述段 + 主要会计数据），共 600 块 | 免费 | `data/graph/blocks.jsonl` |
| ② 抽实体与关系 | LLM 按 13 类实体抽，关系带"为什么相关"+强度分，抽完再问一轮补漏 | **1,772 次调用** | **12,498 实体 / 21,942 关系** |
| ③ 合并描述 | 同一实体在几十块里的描述合成一段（只做提及 ≥3 次的 745 个） | 125 次调用 | 744 个实体有描述 |
| ④ 社区聚类 | **Leiden**（igraph + leidenalg），先切到叶子社区 ≤10 实体，再逐层往上聚 | 免费 | 4 层：224 → 226 → 252 → 6361 |
| ⑤ 社区报告 | 每个社区一份报告，要求"每条要点必须带数字和口径" | 594 次调用 | **1,244 份报告** |
| ⑥ 向量化入库 | 实体描述 + 社区报告编码成向量（放 GPU 上跑，68s + 24s） | 免费（本机模型） | 12,498 + 1,244 条向量 |

两个针对性设计（都是课程讲义点过的问题）：

1. **公司名强制用规范全称**——否则"茅台"和"贵州茅台"会变成两个节点，图就散了；

2. **数值必须写成「指标=数值（期间/口径）」**——课程讲义那份图谱有 21,547 个光秃秃的数值节点，

最后"只知道有个数，不知道是谁的、哪个口径"。我们这次正因为带了口径，才能顺着边把 12 家的数字提取出来。

## 三、三道全景题 × 五条路线（核心结果）

判分：答案里**公司名和数值同时出现**才算覆盖到这家公司。

| 路线 | Q9 营收增速最高/最低 | Q10 净利下滑的公司 | Q11 行业挑战 + 各家应对 | 用时 |
|---|---|---|---|---|
| A 向量 RAG · top-6 | 2/12 | 2/12 | 提到 5/12 家 | 20–54s |
| B 向量 RAG · 重排 4 块 | 2/12 | 3/12 | 提到 4/12 家 | 45–108s |
| C 离线指标表（知识侧预处理） | **12/12** | **12/12** | **12/12** | **5–14s** |
| D GraphRAG · global（读社区报告 map-reduce） | 4/12 | 7/12 | 提到 6/12 家 | 164–329s |
| E GraphRAG · 指标聚合（顺图上的边取数） | **12/12** | **12/12** | **12/12** | **18–27s** |

> E 那一行是**补全指标边之后**的成绩（补之前是 9/12 和 7/12）。怎么补的见下面第四节——

> 这一段比结论本身更值得看：**同样的方法，覆盖度不够就是答不全，补上就满分。**

（A、B 就是作业 A 原来的两条检索路线；C 是我们在深度二末尾补的那张跨公司指标表。）

**Q11 那一列要单独说明**：这里的数字只统计"答案里提到了几家公司"，

而 Q11 真正考的是**有没有答出"行业挑战 + 各家应对"**。按这个标准看：

- **A（向量 RAG）**：给出 5 家公司的具体挑战描述，细节扎实，但只覆盖 5 家；
- **B（重排）**：4 家；
- **C（指标表）/ E（图谱指标边）**：能把 12 家的数字列全，**但它们手里只有指标、没有叙述**，

  答不了"挑战与应对"；

- **D（图谱读社区报告）**：**唯一一条给出了行业级总结的路线**——

  挑战归纳为"深度调整期、需求收缩、存量竞争加剧、渠道库存与动销压力、价格与盈利承压、

  消费理性化与价位分化、原材料成本上涨"，应对归纳为"控量较为价、去库存、产品创新、渠道优化、

  品牌重塑、低度化年轻化、区域深耕 + 文化 IP、压缩费用、数字化与电商"，

  并诚实声明"逐家完整应对未覆盖"。

所以：**数字型全景题靠结构，叙述型全景题靠社区报告**——两条路各有一套用武之地。

## 四、三条结论

### 三•补 为什么第一版只有 9/12、7/12，怎么补到 12/12

第一版 E 路线 Q9 只覆盖 9/12（缺贵州茅台、今世缘、水井坊），Q10 只有 7/12。

查下来**不是方法不行，是图里缺边**：通用抽取只对"关系"负责，同一张主要会计数据表，

它可能抽了"公司→营业收入=…"，却没抽"营业收入同比增减"，于是顺边取数时就缺人。

补法不是重跑全图（那要 2,500 次调用），而是**定向补边**：

1. 对每家公司的「主要会计数据」块，用一份**定死的 schema** 再抽一次

（只要四个指标：营业收入 / 归母净利润 / 扣非归母净利润 / 经营活动现金流，各带数值、期间、口径、同比）；

2. 抽完做一次**完备性校验**：12 家里有几家有"营收同比"这条边?

第一遍补完是 12/12 和 11/12（舍得酒业的表被切到了相邻块，归母净利没抽到）；

3. 针对缺的那家**带上相邻块**再补一次。

补进 135 条边后（12,557 实体 / 22,007 关系），E 路线 **Q9 12/12（18 秒）、Q10 12/12（27 秒）**，

比离线指标表还快一点。

> 这说明一件事：**图谱答全景题的前提是"边建全了"**，而边建没建全是可以用校验脚本量出来的

> （"12 家 × 关键指标"做一张覆盖度矩阵，缺哪格补哪格）。这也是课程讲义那句

> "图谱是通往分散证据的地图"的另一面——**地图缺路，导航就会失败，但这不代表导航方法错。**

**1. 织成网确实有用，但不是万能药。**

向量 RAG 在全景题上是废的（2–3/12），图谱路线把它拉到 4–9/12。

direction 是对的：**答案散在关系里时，必须首先关系抽出来存好。**

而且**只有图谱能回答叙述型全景题**：Q11 里唯一给出"行业挑战 + 应对主线"的完整归纳的就是 D。

**2. 图谱"读报告"不如图谱"读结构"。**

同样是图谱，D（读社区报告做 map-reduce）只到 4/7/6 家，

而 E（顺着「公司→指标=数值（期间/口径）」的边直接取数）到 9/7/12 家。

原因很具体：**社区报告是 LLM 概括过的文本，map-reduce 过程中数字会被压缩、被打分筛掉**；

而结构化的边是原样存下来的。

→ 这正是课程讲义那句"**图谱是通往分散证据的地图，不是证据替代品**"的实测版。

**3. 结构性问题的正解仍是"知识侧预处理"。**

那张离线指标表（C）**12/12 全中且最快（5–14 秒）**，因为它就是为这类问题专门做的结构化表。

换成叙述型的 Q11（行业挑战 + 各家应对），起作用的是图谱的社区报告（D），而不是指标表——

因为"挑战/应对"是关系与叙述，不是数字。

→ 结论同课程讲义：**先问"答案是什么形状"，再决定建表还是建图。**

## 四•补 分题型给答案（实用口径）

| 问题形状 | 该用哪条路 | 本次实测 |
| --- | --- | --- |
| 单公司具体数字 | 向量 RAG（top-6 / 重排） | 作业 A 的 8/10 全对 |
| 跨公司数字聚合（谁最高、谁下滑） | 结构化指标表 / 图谱的指标边 | 12/12（表）、9/12（图谱边） |
| 行业级叙述（普遍面临什么、各家怎么应对） | GraphRAG global 读社区报告 | 唯一给出完整归纳（D） |
| 具体实体的关系（i茅台和经销体系什么关系） | GraphRAG local 沿边扩展 | 见 `query_graph.py --mode local` |

## 五、两个真踩到的工程问题（比结论更有用）

**问题一：把"执行过"当成"成功了"。**

第一轮抽取 600 块时接口一度不稳定，**286 块返回空结果**（山西汾酒、迎驾贡酒、今世缘、口子窖四家均为 50/50 全部为空）。

我的代码重试 3 次失败后返回空结果，**并把空结果写进了"已完成"缓存**——第二次跑时全部被跳过，

图谱直接缺了四家公司，全景题自然答不出来。

修法：**缓存只记成功**（加载时剔除空结果）、重试 3→5 次。补抽后 600/600 全部有内容，

实体从 7,032 涨到 12,498。

**问题二：一次性把 7,032 条文本输入模型。**

第六步最初想在本机 CPU 上编码，代码是"一次把所有文本送进去"，padding 到 1024 token

等于一次前向算 700 万 token，直接卡死。改成分 32 条一批后能跑，但本机 2 分钟才 32 条（要两个多小时）；

最后放到 GPU 上，**12,498 条实体 68 秒、1,244 份报告 24 秒。**

## 六、账（深度三这一块）

| 项目 | 数值 |
| --- | --- |
| 抽取（含补抽） | 1,772 次调用，约 2.8M 输入 + 2.0M 输出 token |
| 合并描述 | 125 次调用 |
| 社区报告 | 594 次调用（另有 650 份复用旧结果） |
| 向量编码 | 0 元（本机模型，只花 GPU 机器 1 分钟） |
| **合计** | 约 **2,500 次调用、5M 级别 token** |

参考课程讲义自己的账：83 份半年报的图谱是 **42,396 次调用、输入 58.9M + 输出 146.1M token。**

我们这次是它的 1/10 量级——这也是课程讲义说的"让模型通读全书的价格"。

## 七、交付物

| 文件 | 内容 |
| --- | --- |
| `data/graph/graph.json` | 图谱本体：12,498 实体、21,942 关系、社区层级 |
| `data/graph/community_reports.jsonl` | 1,244 份社区报告（每层都有，带出处块 id） |
| `data/graph/graph_vectors_*.npy` | 实体/报告的向量（GPU 编码） |
| `graph/*.py` | 六步的完整实现（选材、抽取、合并、聚类、写报告、查询、配图） |
| `outputs/screenshots/A_图谱全景.png` | 图谱配图（节点大小=连接数，颜色=实体类型） |
| `data/graph/panorama_cache.json` | 三道全景题 × 五条路线的完整回答（可逐条核对） |